

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.21

RUNTIME PARAMETERS

 Copy as Markdown

Runtime parameters

Runtime parameters in Serverpod allow you to fine-tune the behavior of the database engine for specific queries or workloads. This can significantly improve performance, especially for complex queries or large datasets.

WARNING

Setting runtime parameters affects PostgreSQL's query planning and execution. Always test different parameter combinations with your specific dataset and query patterns to find the optimal configuration.

INFO

Runtime parameters are only supported for PostgreSQL. Setting runtime parameters on SQLite is a no-op.

Parameter scopes

Runtime parameters can be applied with different scopes:

- **Global scope:** Parameters set at Serverpod startup that affect all database connections and sessions.
- **Transaction scope:** Parameters that only affect queries within a specific transaction.

INFO

Due to connection pooling, runtime parameters cannot be set at the session level as this would create inconsistencies between different database connections. Parameters must be either global (set at startup) or transaction-scoped.



Ask AI

Setting global runtime parameters on startup

To set global runtime parameters that apply to all database sessions, configure them when initializing your Serverpod instance. Add the runtime parameters configuration in the `run` function of your `lib/server.dart` file:

```
import 'src/generated/serverpod.dart';

/// The starting point of the Serverpod server.
void run(List<String> args) async {
  final pod = Serverpod(
    args,
    // Set global runtime parameters that apply to all database sessions.
    runtimeParametersBuilder: (params) => [
      params.vectorIndexQuery(enableIndexScan: true),
      params.hnswIndexQuery(efSearch: 64),
    ],
  );

  // Start the server
  await pod.start();
}
```

Setting transaction-scoped runtime parameters

To set runtime parameters that only apply to queries within a specific transaction, use the `setRuntimeParameters` method on the transaction object:

```
// Transaction scope - only affects current transaction
await session.db.transaction((transaction) async {
  await transaction.setRuntimeParameters((params) => [
    params.hnswIndexQuery(efSearch: 40),
  ]);

  var results = await Document.db.find(
    session,
    where: (t) => t.embedding.distanceCosine(queryVector) < 0.5,
    limit: 10,
    orderBy: (t) => t.embedding.distanceCosine(queryVector),
    transaction: transaction,
  );

  return results;
});
```

These parameters will override any global settings for the duration of the transaction, allowing you to fine-tune query behavior without affecting other transactions or sessions.

Parameter groups

Runtime parameters are set as groups of related settings that control various aspects of query execution. You can use the builder pattern to create these parameter groups, that will always set all parameters in a consistent way.

Existing parameter groups include:

- **Vector index query parameters:** Control the general behavior of index building and querying.
- **HNSW index query parameters:** Control the behavior of HNSW index queries.
- **IVFFLAT index query parameters:** Control the behavior of IVFFLAT index queries.
- **Schema search paths:** Control the search paths for database schemas.

It is also possible to [create custom parameter groups](#).

Combining multiple parameter groups

You can combine different runtime parameter groups at once with the builder callback:

```
await session.db.transaction((transaction) async {
  await transaction.setRuntimeParameters((params) => [
    params.vectorIndexQuery(enableIndexScan: true),
    params.hnswIndexQuery(efSearch: 64),
  ]);

  // Your queries here...
});
```

Creating custom runtime parameters

Custom parameter groups can be created in one of the following ways:

Using `MapRuntimeParameters`

Create an instance of the `MapRuntimeParameters` class, which allows you to set specific parameters by name:

```
import 'package:serverpod/database.dart';

await session.db.transaction((transaction) async {
  // Clear specific parameters within this transaction
  await transaction.setRuntimeParameters((params) => [
    MapRuntimeParameters({
      'param1': null,
      'param2': 2,
      'param3': true,
    }),
  ]);

  // Your queries here...
});
```

```
});
```

Extending `RuntimeParameters`

Extend the base `RuntimeParameters` class directly, overriding the `options` getter:

```
import 'package:serverpod/database.dart';

class CustomRuntimeParameters extends RuntimeParameters {
  final int param1;
  final int param2;

  CustomRuntimeParameters({
    required this.param1,
    required this.param2,
  });

  @override
  Map<String, dynamic> get options => {
    'custom_param': param1,
    'another_param': param2,
  };
}
```

Extending the builder class with custom methods

Create an extension on `RuntimeParametersBuilder` to add custom methods that return instances of your custom runtime parameters class:

```
import 'package:serverpod/database.dart';

extension CustomRuntimeParametersExtension on RuntimeParametersBuilder {
  // Directly with `MapRuntimeParameters`, without a custom class.
  MapRuntimeParameters mapCustomParameters({
    required int param1,
    required int param2,
  }) {
    return MapRuntimeParameters({
      'custom_param': param1,
      'another_param': param2,
    });
  }

  // With a custom runtime parameters class as declared above.
  CustomRuntimeParameters classCustomParameters({
    required int param1,
    required int param2,
  }) {
    return CustomRuntimeParameters(
      param1: param1,
      param2: param2,
    );
  }
}
```

```
    };  
  }  
}
```

This allows you to use your custom parameters with the same builder pattern:

```
await session.db.transaction((transaction) async {  
  await transaction.setRuntimeParameters((params) => [  
    params.mapCustomParameters(param1: 100, param2: 200),  
    params.classCustomParameters(param1: 300, param2: 400),  
    params.hnswIndexQuery(efSearch: 64), // Can combine with built-in parameters  
  ]);  
  
  // Your queries here...  
});
```

Parameter values

Values for runtime parameters can be set to specific types, such as `int`, `double`, `bool`, or `String`. When setting a parameter, you can also set it to `null` to reset it to its default value. Some custom enums, like `IterativeScan`, can also be used to control specific behaviors.

Database schema search paths

You can configure database schema search paths on either global level or within transactions for better schema resolution:

```
await session.db.transaction((transaction) async {  
  // Set custom search paths for schema resolution  
  await transaction.setRuntimeParameters((params) => [  
    params.searchPaths(['my_schema', 'public']),  
  ]);  
  
  // Your queries here will use the custom search paths...  
  
  // Or reset to default search paths  
  await transaction.setRuntimeParameters((params) => [  
    params.searchPaths(), // No arguments resets to default  
  ]);  
  
  // Other queries here with default paths...  
});
```

Vector runtime parameters

When working with vector similarity searches, you can fine-tune query performance by setting runtime parameters that control the behavior of vector indexes.

INFO

Runtime parameters are particularly useful when you need to balance between query speed and result quality. For real-time applications, you might prefer faster but less accurate results, while for analytical workloads, you might prioritize accuracy over speed.

General vector query optimization

Use the builder pattern within transactions to configure general PostgreSQL settings that affect vector query performance:

```
await session.db.transaction((transaction) async {
  // Configure general vector query parameters
  await transaction.setRuntimeParameters((params) => [
    params.vectorIndexQuery(
      enableIndexScan: true,           // Enable/disable index scans
      enableSeqScan: false,           // Enable/disable sequential scans
      minParallelTableScanSize: 1024, // Min number of 8KB chunks to scan before
parallelizing scans
      parallelSetupCost: 1000,         // Estimated cost of launching parallel worker
processes
      maintenanceWorkMem: 65536,      // Memory in KB for operations such as index
creation
      maxParallelMaintenanceWorkers: 2, // Increase to speed up index creation on large
tables
      maxParallelWorkersPerGather: 2,  // Increase to speed up queries without an index
    ),
  ]);

  // Perform vector search with optimized PostgreSQL settings
  var results = await Document.db.find(
    session,
    where: (t) => t.embedding.distanceCosine(queryVector) < 0.7,
    orderBy: (t) => t.embedding.distanceCosine(queryVector),
    limit: 50,
    transaction: transaction,
  );

  return results;
});
```

HNSW index optimization

For queries using HNSW indexes, use the builder pattern within transactions to control search behavior:

```
import 'package:serverpod/serverpod.dart';

await session.db.transaction((transaction) async {
  // Configure HNSW runtime parameters
  await transaction.setRuntimeParameters((params) => [
    params.hnswIndexQuery(
```

```

        efSearch: 40, // Higher values = better recall, slower
search
        iterativeScan: IterativeScan.relaxed, // Relaxed scan for better recall
        maxScanTuples: 20000, // Limit number of tuples to scan
        scanMemMultiplier: 2, // Memory multiplier for scanning
    },
    });

    // Your HNSW queries here...
});

```

IVFFLAT index optimization

For queries using IVFFLAT indexes, use the builder pattern within transactions to control search behavior:

```

await session.db.transaction((transaction) async {
    // Configure IVFFLAT runtime parameters
    await transaction.setRuntimeParameters((params) => [
        params.ivfflatIndexQuery(
            probes: 10, // Number of probes to search (higher =
better recall)
            iterativeScan: IterativeScan.relaxed, // Relaxed scan for better recall
            maxProbes: 20, // Maximum number of probes to use
        ),
    ]);

    // Your IVFFLAT queries here...
});

```

NOTE

The `IterativeScan.strict` mode is not supported for IVFFLAT indexes. Use `IterativeScan.relaxed` or `IterativeScan.off` (default) instead.

Iterative scan modes

The `IterativeScan` enum controls how vector indexes handle scanning:

- `IterativeScan.off`: Disables iterative scanning, using the default `pgvector` behavior.
- `IterativeScan.strict`: Ensures results are in exact order by distance (better precision).
- `IterativeScan.relaxed`: Allows slightly out-of-order results but provides better recall.

Testing with runtime parameters

When writing tests that use runtime parameters, you'll typically want to configure them either globally for the test suite or within specific test transactions. On transaction scope, it is the same as in production code, where you set the parameters within the transaction to ensure they only apply to that specific test case.

Global test configuration

For tests that require specific runtime parameters throughout the test suite, you can configure them using the `withServerpod` test helper:

```
import 'package:test/test.dart';

import 'test_tools/serverpod_test_tools.dart';

void main() {
  withServerpod(
    'Given a server with vector runtime parameters',
    runtimeParametersBuilder: (params) => [
      params.vectorIndexQuery(enableIndexScan: true),
      params.hnswIndexQuery(efSearch: 100),
    ],
    (sessionBuilder, endpoints) {
      // Tests here will use the configured runtime parameters
    },
  );
}
```

This is the same as configuring the runtime parameters in the `run` function, but it allows you to set them specifically for your test environment.

Ensuring vector extension for runtime parameter tests

When testing runtime parameters, it is possible to find unexpected behavior of some parameters returning `null` if queried before the vector extension is loaded. To avoid this and ensure consistent behavior, call the `ensureVectorLoaded` method before running such tests.

This is not necessary for production code. The only use case that it does make a difference is when explicitly querying runtime parameters that are related to vector operations before the vector extension is loaded:

```
import 'package:serverpod_test/serverpod_test.dart';

void main() {
  withServerpod('Given withServerpod without runtimeParametersBuilder',
    (sessionBuilder, endpoints) {
      var session = sessionBuilder.build();

      test(
        'when querying runtime parameters globally then expect default values',
        () async {
          // Ensure vector extension is loaded before, so that all vector runtime
          // parameters have their default values set correctly.
          await session.db.ensureVectorLoaded();

          var hnswCheckQuery = HnswIndexQueryOptions().buildCheckValues();
          var hnswResult = await session.db.unsafeQuery(hnswCheckQuery);
        }
      );
    }
  );
}
```

```
var hnsRow = hnsResult.first.toColumnMap();

// This would return null if the vector extension is not loaded, making
// the expect unpredictable depending on previous tests/operations.
expect(hnsRow['hns_ef_search'], '40');

await session.db.transaction((transaction) async {
  await transaction.setRuntimeParameters((params) => [
    params.hnsIndexQuery(efSearch: 200),
  ]);

  // Your test queries here...
});
});
});
}
```